

Module 6 — Identity, Authentication & Authorisation

Three distinct concepts, three distinct attack surfaces, three distinct defensive layers

A07 ID & Auth Failures | A01 Broken Access Control | BBC Academy / JBI Training

	Concept & Definition	What the Server Must Establish	Attack Vectors (flaw refs from ShopCo / Juice Shop)	Defensive Controls
IDEN	1. IDENTITY <i>"Who claims to be making this request?"</i>			
	Identity is the claimed name or identifier a user presents to the system — before any verification has taken place. It is an assertion, not a proof. The server has no grounds to trust it until authentication succeeds.	A unique, non-guessable identifier per user. Never derived from user-supplied input alone. In practice: a user_id from a database record, not a username or email address that the user controls.	Attack vectors: ▶ Spoofing — claiming a different user's identity by forging the username or user_id in a request parameter [ShopCo Flaw 7 (IDOR)] ▶ Username enumeration — probing login error messages to confirm which identities exist in the system [Module 10 OSINT] ▶ Account impersonation — using a known email/username format to target specific individuals [Juice Shop: Login Admin]	Defensive controls: ✓ Use immutable, server-generated user_id — never accept identity from client-controlled parameters ✓ Return identical error messages whether the username exists or not (prevents enumeration) ✓ Never expose sequential or predictable user IDs in URLs or API responses ✓ Log every identity claim attempt with timestamp, source IP, and outcome
	<i>Identity must be verified before a session is created ▼ verification is Authentication</i>			
AUT	2. AUTHENTICATION (AuthN) <i>"Can this identity prove it is who it claims to be?"</i>			
	Authentication is the process of verifying an identity claim using one or more factors: something you know (password), something you have (TOTP device), or something you are (biometric). Only after successful authentication should a session token be issued.	A valid credential bound to the claimed identity: a correctly hashed password match, a valid TOTP code, a signed assertion from a trusted IdP. The credential must be verified server-side — never in client-side JavaScript.	Attack vectors: ▶ Hash cracking (offline) — extract MD5/SHA1 hashes from DB, crack with HashCat + rockyou.txt in seconds [ShopCo Flaw 3] ▶ Credential stuffing — replay breached username:password pairs at scale against the login endpoint [Juice Shop: Login with breached credentials] ▶ Session forgery — forge a valid session cookie using a known or weak secret key [ShopCo Flaw 1] ▶ Cookie stealing (via XSS) — inject script that reads document.cookie and	Defensive controls: ✓ bcrypt / scrypt / Argon2 with work factor ≥12 — never MD5, SHA1, or unsalted hashes ✓ HttpOnly + Secure + SameSite=Lax cookie flags — prevents JS access and transmission over HTTP ✓ Session secret from os.urandom(32) or environment variable — never hardcoded ✓ Regenerate session ID on every privilege

		exfiltrates the session token [Module 6 & 7] ▶ JWT alg:none attack — remove the signature from a JWT and set algorithm to "none" [Flask Supplement M6]	change (login, role change, logout) ✓ Rate limiting: max 5 login attempts per IP per minute (Flask-Limiter) ✓ MFA for privileged accounts — TOTP or FIDO2 hardware key
--	--	---	--

Authentication proves identity and issues a session token ▼ that token is then used for Authorisation

AUT

3. AUTHORISATION (AuthZ)

"Is this authenticated identity permitted to perform this action on this resource?"

Authorisation happens after authentication. It is the decision about what an authenticated user is permitted to do. Authentication asks "are you who you say you are?" — Authorisation asks "are you allowed to do this?" They are entirely separate checks and must both be enforced server-side on every request.

A verified mapping between the authenticated user's identity and the permissions granted to that identity. The server must check: (1) is the user authenticated? (2) does their role/attribute permit this action on this specific resource? Both checks must pass — "authenticated" is not the same as "authorised".

Attack vectors:
 ▶ IDOR — authenticated user changes a resource ID in the URL to access another user's data **[ShopCo Flaw 7 / Juice Shop: View Basket]**
 ▶ Privilege escalation — forging or manipulating a role/permission token to gain higher access **[ShopCo Flaw 5]**
 ▶ Missing function-level access control — admin endpoints not protected because "users won't find them" **[ShopCo Flaw 6 / Juice Shop: Admin Panel]**
 ▶ Forced browsing — directly navigating to URLs that the UI doesn't link to but the server doesn't protect **[Juice Shop: Confidential Document]**

Defensive controls:
 ✓ Check authorisation server-side on every request — client-side hiding is not a control
 ✓ Deny by default: if no explicit permission is granted, the action is denied
 ✓ Resource-level checks: if user.id != resource.owner_id and not admin → 403 Forbidden
 ✓ @login_required on every protected route — never rely on obscurity
 ✓ Principle of Least Privilege: grant only the minimum permissions needed for the role
 ✓ RBAC or ABAC implemented server-side — roles stored in DB, never in session cookie

The Three Most Common Confusions — All of Which Cause Real Breaches

Common Confusion 1	Common Confusion 2	Common Confusion 3
"Logged in means authorised." Being authenticated proves identity. It says nothing about what resources or actions are permitted. Every resource access requires a separate authorisation check — even for authenticated users.	"HTTPS means the session is secure." HTTPS protects the token in transit. It does not protect a token with no HttpOnly flag (stealable by XSS), a weak session secret (forgeable), or a missing expiry (persistent after logout). Transport and session security are separate concerns.	"The UI doesn't show it, so users can't access it." Security through obscurity is not authorisation. Any URL or API endpoint reachable by HTTP must enforce its own authorisation check server-side, regardless of whether the UI links to it. Juice Shop's /administration panel is a live demonstration of this failure.

The test: for any endpoint in your application, can you answer all three questions independently? (1) **How does the server know who this is?** (2) **How does the server verify they are who they claim?** (3) **How does the server decide whether this identity is permitted to perform this action on this specific resource?** **If any answer is "it trusts the client" — that is the vulnerability.**